

Lépésről lépésre: a 3 RoboDK mérési feladat megoldása és dokumentálása

Ez az útmutató végigvezet a három mérési feladaton. Minden kattintás, beírandó érték, mérés és értelmezés szerepel benne, így önállóan meg tudod csinálni, és meg is tudod védeni.

Két út van, és érdemes mindkettőt használni:

	Mire jó	Mikor használd
Kézi út (ez az útmutató)	A RoboDK-ban magad építed fel a cellát és a programokat	Ezzel tanulsz; a vizsgán, védésen ezt kell tudnod
Automatikus út (<code>robodk_scripts/cella_epito.py</code>)	A szkript felépíti ugyanezt a mintamegoldást	Ellenőrzésre, összehasonlításra, vagy ha elakadsz

A mérést és a jegyzőkönyvet mindkét esetben ugyanazok a szkriptek készítik el (`meresek_rogzitese.py`, `jegyzokonyv_keszito.py`), ezért a dokumentáció egyforma lesz.

Időigény: az előkészület kb. 30 perc. Kézzel feladatonként 1–2 óra (az elsőnél több, mert akkor ismered meg a RoboDK-t), mérés és jegyzőkönyv kb. 1 óra.

Tartalom

- 0. Előkészületek (egyszer kell megcsinálni)
- 1. feladat – Pick & Place (UR5e)
- 2. feladat – Négyzet alakú pálya (ABB IRB 120)
- 3. feladat – Precíziós mozgás tiltott zónával (KUKA LBR iiwa 7)
- 4. Mérés és jegyzőkönyv
- 5. Gyakori hibák
- 6. Mit írsz az értékelésbe?

0. Előkészületek (egyszer kell megcsinálni)

0.1 Programok

1. **RoboDK:** robodk.com/download. Ha az egyetemnek van licence, azt használd. Különben 30 napig próbaverzió fut, utána a mentés és az export korlátozott lehet. Egyetemi e-mail-címmel a RoboDK-tól kérhetsz hosszabbítást.
2. **Ez a csomag:** a GitHubon **Code → Download ZIP**, majd csomagold ki, például ide:
`C:\RoboDK_feladatok\eszkozok\`.
3. **Python a jegyzőkönyvhöz** (3.8 vagy újabb, python.org). Telepítéskor pipáld be az *Add Python to PATH* opciót. Utána a kicsomagolt mappában nyiss parancssort (a címsorba írd: `cmd`, majd Enter), és futtasd:

```
python -m pip install -r requirements.txt
```

A RoboDK-n belül futó szkriptekhez semmit nem kell telepíteni, mert a RoboDK hozza a saját Pythonját.

0.2 Mappák: minden feladat külön állomás

```
C:\RoboDK_feladatok\  
├─ eszkozok\           ← ez a csomag (szkriptek, útmutató)  
├─ Feladat1.rdk        ← 1. feladat állomása  
├─ Feladat2.rdk  
├─ Feladat3.rdk  
├─ Feladat1_dokumentacio\ ← a szkriptek hozzák létre (lépések, képek, mérések, feladat.txt)  
├─ Feladat2_dokumentacio\  
└─ Feladat3_dokumentacio\
```

Fontos: az állomást az első lépés után azonnal mentsd el (**File → Save Station as**, pl. `Feladat1.rdk`). A szkriptek az állomás neve alapján, az `.rdk` mellé mentenek. Így a 3 feladat adatai nem keverednek.

0.3 RoboDK-alapok, mielőtt nekikezdesz (5 perc gyakorlás)

Mit akarsz	Hogyan
Kijelölés	bal egérgomb
Nézet forgatása	jobb egérgombot nyomva tartva húzd
Nézet eltolása	középső gombot (görgőt) nyomva tartva húzd
Nagyítás	görgő
Keret vagy objektum mozgatása egérrel	Alt + húzás
Robot TCP-jének mozgatása egérrel	Alt + Shift + húzás
Robot pontos helyzetbe állítása	dupla kattintás a robotra → a robot ablakában a <i>Tool ... w.r.t. Reference</i> mezőbe írd be az X, Y, Z, Rx, Ry, Rz értékeket
Célpont (target) felvétele	Program → Teach Target (Ctrl+T): a robot aktuális helyzetét rögzíti az aktív referencia keretben
Célpont koordinátáinak módosítása	jelöld ki a célpontot → F3 (Target Options)
Átnevezés	jelöld ki → F2
Program futtatása (szimuláció)	dupla kattintás a programra
Pálya ellenőrzése	jobb klikk a programra → Check path (F5)

Mit akarsz	Hogyan
Pálya + ütközés ellenőrzése	Shift+F5
Mentés	Ctrl+S

Laptonon, egér nélkül: jobb klikk a 3D nézetben, ott is megtalálod a forgatás, eltolás és nagyítás parancsokat.

0.4 A szkriptek használata

Mindegyik szkriptet ugyanúgy indítod: **húzd be a .py fájlt a RoboDK ablakába** (vagy **File → Open**). A fában megjelenik a nevével, és **dupla kattintással fut**. A tartalma az állomásba mentődik, így egyszer kell behúzni.

Szkript	Mikor futtasd	Mit csinál
lepes_rogzitese.py	minden fontos lépés után (a 📷 jelnél)	Bekér egy rövid leírást, és képernyőképet ment. Ebből lesz a jegyzőkönyv „A megvalósítás lépései” fejezete.
doboz_letrehozasa.py	ha dobozt (asztal, munkadarab, tiltott zóna) kell létrehozni	Megadott méretű, színű dobozt tesz a kijelölt keretbe
meresek_rogzitese.py	a kész megoldáson, sebességváltozatoként akár többször	Minden mérést kiment (idők, utak, sebességek, pontosság, ütközés, tiltott zóna)
cella_epito.py	ha a mintamegoldást akarod (automatikus út)	Felépíti a teljes cellát és a programokat

Mielőtt képet készítesz, állítsd be a 3D nézetet. A szkript azt menti, amit a képernyőn látsz.

0.5 Fontos szakmai háttér: mit mér a szimuláció, és mit nem?

Ezt mindhárom feladat értékelésénél fel fogod használni.

- A RoboDK **ideális kinematikai modellel** számol. A **megálló (pontos, „fine”) célpontokat** a robot **sebességtől függetlenül pontosan eléri**: az eltérés gyakorlatilag 0 mm, a mérés felbontásán belül. A szimuláció **determinisztikus**, ezért ugyanaz a program mindig ugyanazt adja. Az ismételhetőség tehát szimulációban ideális.
- Eltérés a szimulációban csak **lekerekítéskor** (*Rounding / blending / zone*) jelenik meg: ilyenkor a robot nem áll meg a célpontban, hanem „levágja” a sarkot. Az eltérés nagyságát a lekerekítés mértéke szabja meg, nem a sebesség.
- Valódi robotnál** a nagyobb sebesség és gyorsulás nagyobb pályahibát okoz: a hajtások késnek, a sarkokban túllendül a robot, a karok rugalmasan deformálódnak. A pozicionálási ismételhetőséget az adatlap adja meg (ISO 9283). Például UR5e: ±0,03 mm, ABB IRB 120: 0,01 mm. Ezeket az értékeket mindig ellenőrizd az adott robot adatlapján.
- A **ciklusidő** a RoboDK becslése a beállított sebességek és gyorsulások alapján. A valódi robotnál ettől kissé eltérhet.

A jegyzőkönyvbe tehát nem az kerül, hogy „gyorsabban pontatlanabb lett”, hanem a fenti, mérésekkel alátámasztott gondolatmenet.

1. feladat – Pick & Place (UR5e)

Cél: a robot vegyen fel egy munkadarabot, és tegye át egy másik helyre. MoveJ és MoveL mozgással, három sebességgel.

Robot: Universal Robots UR5e (modern, 6 tengelyes kollaboratív robot; a „movej/movel” is UR-kifejezés).

1.1 A megoldás terve

Minden koordináta az **Asztal** referencia keretben értendő. A szerszám lefelé néz, ezért a célpontok orientációja minden esetben **Rx = 180°, Ry = 0°, Rz = 0°**.

Elem	Típus	Adatok
Megfogo	szerszám	TCP: Z = 160 mm a karimától (a munkadarab közepénél fog)
Asztal	referencia keret	a robot bázisához képest X = 400, Y = 0, Z = 0 mm
Asztallap	doboz	500 × 800 × 20 mm, hely: (0; 0; -20)
Munkadarab	doboz	50 × 50 × 50 mm, hely: (0; -200; 0)
Lerakohely	doboz (jelölő)	70 × 70 × 1 mm, hely: (0; 200; 0)

Célpont	X	Y	Z	Megjegyzés
Home	-50	0	300	kiinduló és végpont
Felvetel_felett	0	-200	125	100 mm-rel a megfogási pont fölött
Felvetel	0	-200	25	a kocka közepe (50 mm magas kocka)
Lerakas_felett	0	200	125	
Lerakas	0	200	25	

Sebességszint	lineáris [mm/s]	lin. gyorsulás [mm/s ²]	csukló [°/s]	csuklógyorsulás [°/s ²]
lassu	100	250	30	60
kozepes	300	800	90	180
gyors	800	2000	180	400

Programok: PP_MoveJ_lassu , PP_MoveJ_kozepes , PP_MoveJ_gyors , PP_MoveL_lassu , PP_MoveL_kozepes , PP_MoveL_gyors . A MoveJ- és a MoveL-változat csak a szabad mozgásokban (Home → felvétel fölé, átvitel, vissza Home-ba) különbözik. A megközelítés és az emelés mindig MoveL, egyenesen függőlegesen.

1.2 Lépésről lépésre

1. **Új állomás:** File → New Station, majd azonnal File → Save Station as... → **Feladat1.rdk** .
2. **Robot:** File → Open online library (Ctrl+Shift+O). Szűrj a Universal Robots gyártóra, válaszd a UR5e-t, majd Open. A robot megjelenik az állomásban.
3. **A szkriptek behúzása:** húzd be a lepes_rogzitese.py , a doboz_letrehozasa.py és a meresek_rogzitese.py fájlt a RoboDK-ba. 📁 **Lépés rögzítése:** dupla katt a lepes_rogzitese-re → „UR5e robot betöltése az online könyvtárból”.
4. **Szerszám:**
 - **a) Valódi megfogóval:** az online könyvtárban keress egy megfogót (pl. Robotiq 2F-85), **Open**. Ha nem a robotra került, a fában húzd rá a robotra.
 - **b) Egyszerűen:** jelöld ki a robotot, majd **Program → Add Empty Tool**. Kattints duplán az új szerszámmra, a TCP-nél add meg a **Z = 160** értéket, és nevezd át (F2) **Megfogo** -ra.

📁 „Megfogó hozzáadása, TCP: Z = 160 mm”.

5. **Referencia keret:** jelöld ki a fában a robot bázisát (*UR5e Base*), majd **Program → Add Reference Frame**. Nevezd át (F2) **Asztal** -ra, kattints rá duplán, és állítsd be: **X = 400, Y = 0, Z = 0**, a forgatások 0.
6. **Asztal és munkadarab:** jelöld ki az **Asztal** keretet, és futtasd a **doboz_letrehozasa** szkriptet háromszor:
- **Asztallap** : 500 × 800 × 20, hely (0; 0; -20), szürke
 - **Munkadarab** : 50 × 50 × 50, hely (0; -200; 0), kék
 - **Lerakohely** : 70 × 70 × 1, hely (0; 200; 0), zöld

(Alternatíva: **Tools → Add-in Manager** (Shift+A) → a *Shapes/Components* add-in, Box fül.)  „Asztal referencia keret, asztallap, munkadarab és lerakóhely elhelyezése”.

7. **Célpontok:** kattints duplán a robotra. A robot ablakában állítsd az aktív szerszámot **Megfogo** -ra, az aktív referenciát **Asztal** -ra. Ezután minden célpontnál:
1. A *Tool w.r.t. Reference* mezőkbe írd be a koordinátákat a fenti táblázatból ($R_x = 180, R_y = 0, R_z = 0$), és nyomj Entert. A robot odaáll.
 2. **Program → Teach Target** (Ctrl+T), majd nevezd át (F2) a táblázat szerint.

Sorrend: **Home** , **Felvetel_felett** , **Felvetel** , **Lerakas_felett** , **Lerakas** . Ha a robot nem tud odaállni, a pont nem elérhető. Ilyenkor tedd közelebb az **Asztal** keretet (pl. $X = 350$).  „Célpontok felvétele (**Home**, **Felvetel_felett**, **Felvetel**, **Lerakas_felett**, **Lerakas**)”.

8. **Az első program (**PP_MoveJ_kozepes**):** jelöld ki a robotot, **Program → Add Program**, nevezd át (F2). Az utasításokat ebben a sorrendben add hozzá. Mindig az éppen kijelölt utasítás után kerül az új.

#	Menü	Beállítás
1	Program → Simulation Event Instruction	<i>Set object position (relative)</i> : a Munkadarab (a mostani helyén). Minden futás elején visszateszi a kockát.
2	Program → Set Speed Instruction	lineáris 300 mm/s, lin. gyorsulás 800 mm/s ² , csukló 90 °/s, csuklógyorsulás 180 °/s ² (mindegyiket pipáld be)
3	jelöld ki a Home célpontot → Program → Move Joint Instruction	
4	Felvetel_felett → Move Joint Instruction	
5	Felvetel → Move Linear Instruction	függőleges megközelítés
6	Program → Pause Instruction	300 ms (a megfogó zárási ideje)
7	Program → Simulation Event Instruction	Attach object , szerszám: Megfogo
8	Felvetel_felett → Move Linear Instruction	emelés
9	Lerakas_felett → Move Joint Instruction	átvitel
10	Lerakas → Move Linear Instruction	
11	Program → Pause Instruction	300 ms
12	Program → Simulation Event Instruction	Detach object → a darab az Asztal keretre kerül
13	Lerakas_felett → Move Linear Instruction	
14	Home → Move Joint Instruction	

Ha egy mozgásutasítás rossz célpontozhoz kötődött: jobb klikk az utasításon → *Target linked* lista. **Kattints duplán a programra:** a robot áteszi a kockát. A ciklusidő a jobb alsó sarokban jelenik meg.  „PP_MoveJ_kozepes program: megközelítés, megfogás, átvitel, elengedés”.

9. **Ellenőrzés:** kapcsolod be az ütközésvizsgálatot (**Tools → Check collisions**), majd jobb klikk a programra → **Check path and collisions** (Shift+F5). Ha piros elemet látsz, ütközés van. Az ütközési térképet a **Tools → Collision map** (Shift+X) mutatja.

10. **Sebességváltozatok:** jelöld ki a programot → **Ctrl+C**, jelöld ki az állomást → **Ctrl+V**, majd nevezd át (F2) `PP_MoveJ_lassu`-ra. Kattints duplán a *Set Speed* utasításra, és írd át a lassú értékekre. Ugyanígy készül a `PP_MoveJ_gyors`.
11. **MoveL-változatok:** másold a `PP_MoveJ_kozepes`-t → `PP_MoveL_kozepes`. A 4., 9. és 14. utasításon jobb klikk → váltsd **Linear Move**-ra. Ebből készíts lassú és gyors másolatot is. 🌟 „Sebességváltozatok és MoveL-változatok létrehozása”.
12. **Opcionális, az ismételhetőséghez:** a `PP_ismetles_3x` programban a ciklus háromszor ismétlődik oda-vissza. Ezt a `cella_epito.py` automatikusan elkészíti. Kézzel: másold egymás után a mozgásblokkot, a visszaútnál felcserélt felvételi és lerakási ponttal.
13. Futtasd le mindegyik programot egyszer, majd mentsd: **Ctrl+S**.

1.3 Mérés

Futtasd a `meresek_rogzitese`-t. Megnevezésnek írd be például: „végleges, 3 sebesség × MoveJ/MoveL”. A szkript mind a 6–7 programot egyszerre méri.

1.4 Hol találsz a mérendő adatokat a jegyzőkönyvben?

Mérendő adat	Hol van	Mit nézz
Idő a felvételtől a lerakásig	programonként a fő eredmények táblázatában: <i>Felvételtől lerakásig</i>	a felvételi pont elhagyásától a lerakási pont eléréséig eltelt idő
A mozgás során megtett út	<i>TCP pályahossz</i> , és a <i>Célpontok elérése</i> táblázat <i>Úthossz</i> oszlopa	MoveL-lel rövidebb, mert egyenes
A robot maximális sebessége	<i>Legnagyobb TCP sebesség</i> , a csuklókra a <i>Csuklóstatisztika</i> (Max ω)	MoveJ-nél a TCP gyorsabb is lehet a beállított lineáris sebességnél
MoveJ és MoveL végrehajtási idejének különbsége	1.7 <i>Programok összehasonlítása</i> : MoveJ–MoveL táblázat és grafikon	sebességszintenként, s-ban és %-ban
Különböző sebességek összehasonlítása	1.7 <i>A programok ciklusideje</i> grafikon és táblázat	
Ütközésmentesség, pontosság, ismételhetőség	<i>Ütközésvizsgálat</i> ; <i>Legnagyobb eltérés a megálló célpontokban</i> ; <i>Ugyanazon célpont ismételt elérésének eltérése</i>	a sikerességi feltétel igazolása

1.5 Várható eredmények és értelmezésük

- **MoveJ vs. MoveL:** a MoveJ általában **gyorsabb**, mert minden csukló egyszerre, a saját csuklósebességével mozog. A TCP pályája ilyenkor ív. A MoveL-nél a TCP egyenesen halad a beállított lineáris sebességgel, így **rövidebb utat** tesz meg, de hosszabb lehet az idő, és egyes helyzetekben szingularitás közelébe kerülhet.
- **Sebesség:** kétszeres sebesség nem jelent fele ciklusidőt. A gyorsítás és a lassítás szakaszai, valamint a rögzített várakozások (2×300 ms) nem rövidülnek arányosan.
- **Pontosság, ismételhetőség:** szimulációban 0 mm, a mérési felbontáson belül (lásd 0.5). A valódi UR5e ismételhetősége az adatlap szerint $\pm 0,03$ mm.

2. feladat – Négyzet alakú pálya (ABB IRB 120)

Cél: a végrehajtó szerv (egy „toll”) kövessen egy 150×150 mm-es négyzetet lassan és gyorsan, és mérd meg a szakaszidőket, a tengelyek elfordulását és a pályakövetés pontosságát.

Robot: ABB IRB 120-3/0.6 (klasszikus 6 tengelyes csuklós ipari robot, sok egyetemi laborban megtalálható).

2.1 A megoldás terve

A koordinátarendszer használata itt a lényeg. A négyzet pontjait a **PaLya** keretben adod meg, amelynek origója a négyzet közepe. Ha a keretet elmozgatod, a teljes pálya vele mozog, a programhoz nem kell nyúlni.

Elem	Adatok
Toll (szerszám)	TCP: Z = 120 mm
Pálya (keret)	a robot bázisához képest X = 330, Y = 0, Z = 0 mm
Rajzlap (doboz)	297 × 210 × 1 mm (A4), hely (0; 0; -1), fehér

Célpont	X	Y	Z
Home	0	0	200
P1_felett	-75	-75	50
P1	-75	-75	0
P2	75	-75	0
P3	75	75	0
P4	-75	75	0

Az orientáció mindenhol $R_x = 180^\circ$, $R_y = 0^\circ$, $R_z = 0^\circ$.

Program: Set Speed → MoveJ Home → MoveJ P1_felett → MoveL P1 → MoveL P2 → MoveL P3 → MoveL P4 → MoveL P1 → MoveL P1_felett → MoveJ Home .

Program	Lineáris sebesség	Lekerekítés
Negyzet_lassu	50 mm/s (gyorsulás 200 mm/s ²)	nincs (-1, pontos sarkok)
Negyzet_gyors	500 mm/s (gyorsulás 2000 mm/s ²)	nincs
Negyzet_lassu_lekerekített (kiegészítő kísérlet)	50 mm/s	10 mm a P2, P3, P4 sarkokban
Negyzet_gyors_lekerekített (kiegészítő kísérlet)	500 mm/s	10 mm

2.2 Lépésről lépésre

- File → New Station**, mentés: Feladat2.rdk . Töltsd be az **ABB IRB 120-3/0.6**-t az online könyvtárból. Húzd be a szkripteket. 📎
- Szerszám:** jelöld ki a robotot → **Program → Add Empty Tool** → dupla katt → TCP **Z = 120** → átnevezés: Toll .
- Keret:** jelöld ki az **ABB IRB 120 Base** keretet → **Program → Add Reference Frame** → PaLya , X = 330.
- Rajzlap:** jelöld ki a PaLya keretet → doboz_letrehozása : Rajzlap , 297 × 210 × 1, hely (0; 0; -1), fehér. 📎 „Toll szerszám, PaLya keret a négyzet közepén, rajzlap”.
- Célpontok:** a robot ablakában aktív szerszám Toll , aktív referencia PaLya . Minden pontnál írd be a koordinátákat ($R_x = 180$), majd Ctrl+T és F2. 📎 A koordinátarendszer bemutatásához a robot ablakában figyeld meg, hogyan változik ugyanaz a pont, ha a referenciát PaLya helyett a robot bázisára állítod. Ez jó ábra a jegyzőkönyvbe. 📎
- Program Negyzet_lassu : Program → Add Program**, majd a fenti sorrendben a mozgásutasítások. Az elejére egy **Set Speed** kell (50 mm/s, 200 mm/s²). A **Set Rounding** alapértéke pontos (-1). Dupla katt: a toll körbejárja a négyzetet. 📎

7. **Negyzet_gyors** : másolat (Ctrl+C / Ctrl+V), a Set Speed legyen 500 mm/s és 2000 mm/s².
8. **Kiegészítő kísérlet, lekerekítés**: másold a **Negyzet_lassu**-t → **Negyzet_lassu_lekerekített**. A MoveL P2 elé tegyél egy **Program → Set Rounding Instruction**-t **10 mm**-rel, az utolsó MoveL P1 elé pedig egy másikat **-1**-gyel. Így a P1-ben a robot pontosan megáll. Ugyanígy készül a gyors változat. 📷
9. Ellenőrzés (F5) és mentés.

2.3 Hol találod a mérendő adatokat?

Mérendő adat	Hol van
Az egyes pályaszakaszok végrehajtási ideje	programonként a <i>Célpontok elérése</i> táblázat <i>Szakaszidő</i> oszlopa (P1→P2, P2→P3, P3→P4, P4→P1)
A kezdő- és végpozíció közötti eltérés	<i>Ugyanazon célpont ismételt elérésének eltérése</i> (a P1 a kör elején és végén), valamint <i>A program kezdő- és végpontjának távolsága</i>
A tengelyek elfordulása	<i>A tengelyek elfordulása szakaszonként</i> táblázat ($\Delta J1... \Delta J6$ és $\Sigma \Delta $), <i>Csuklószögek az idő függvényében</i> grafikon, <i>Csuklóstatistika</i>
A sebesség hatása a pályakövetés pontosságára	<i>2.7 Programok összehasonlítása: Max eltérés, megálló és Max eltérés egyenestől</i> oszlop, valamint a felülnézeti grafikonok

2.4 Várható eredmények és értelmezésük

- **Szakaszidők**: a négy oldal azonos hosszú (150 mm), ezért a szakaszidők közel egyformák. Minden oldalon gyorsít, majd lassít a robot. A gyors változatban a szakaszidő nem tizedére csökken: 500 mm/s-nál a 150 mm-es oldal nagy részén a robot gyorsít vagy lassít.
- **Kezdő- és végpont**: szimulációban 0 mm. A valódi robotnál ezt az ismételhetőség határozza meg (ABB IRB 120: 0,01 mm az adatlap szerint, ellenőrizd).
- **Tengelyek**: a négyzet egyes oldalain más-más tengelyek dolgoznak. Az X irányú oldalaknál inkább J2/J3/J5, az Y irányúaknál J1 és J6. Ezt a ΔJ -táblázatból olvasd ki, és írd le a saját méréseid alapján.
- **Sebesség és pontosság**: pontos sarkokkal a pálya lassan és gyorsan is ugyanaz, 0 mm eltéréssel. Ez a szimuláció ideális voltából adódik. Lekerekítéssel a sarkokban néhány mm eltérés jelenik meg, és ez lassan és gyorsan ugyanakkora. **A pontosságot a szimulációban a lekerekítés rontja, nem a sebesség.** A valódi robotnál a nagy sebesség a hajtások késése és a túllendülés miatt a sarkoknál is eltérést okozna (lásd 0.5).

3. feladat – Precíziós mozgás tiltott zónával (KUKA LBR iiwa 7)

Cél: a robot egy virtuális műteti területen három pontot érjen el sorban úgy, hogy a tiltott zónába nem lép be. Lassú és gyors sebességgel.

Robot: KUKA LBR iiwa 7 R800. Ez a 7 tengelyes, érzékeny robot az orvosi célú **KUKA LBR Med** alapja. Ha a 7 tengely gondot okoz, UR3e vagy UR5e is jó.

3.1 A megoldás terve

Elem	Adatok
Muszer (szerszám)	TCP: Z = 180 mm (vékony műszer)
Muteti_terulet (keret)	a robot bázisához képest X = 500, Y = 0, Z = 0 mm
Mutoasztal (doboz)	300 × 300 × 20 mm, hely (0; 0; -20), világoskék
Tiltott_zona (doboz)	60 × 60 × 60 mm, hely (0; -60; 0), piros, átlátszó

Célpont	X	Y	Z	Szerep
Kiindulas	0	0	150	kiindulási pont
C1_felett	-80	-60	100	biztonsági pont, 40 mm-rel a zóna teteje fölött
C1	-80	-60	0	1. célpont
C2_felett	80	-60	100	
C2	80	-60	0	2. célpont
C3_felett	0	70	100	
C3	0	70	0	3. célpont

Orientáció: Rx = 180°, Ry = 0°, Rz = 0°.

Az akadálykerülés elve: a tiltott zóna a C1 és a C2 között van. A robot ezért a célpontokat **függőlegesen, felülről** közelíti meg, a pontok között pedig a **100 mm-es biztonsági magasságon** halad. A zóna 60 mm magas, így 40 mm biztonsági távolság marad.

Program: Set Speed → MoveJ Kiindulas → MoveL C1_felett → MoveL C1 → MoveL C1_felett → MoveL C2_felett → MoveL C2 → MoveL C2_felett → MoveL C3_felett → MoveL C3 → MoveL C3_felett → MoveL Kiindulas . A precíziós mozgásnál mindenhol MoveL kell, mert így kiszámítható, egyenes a pálya.

Program	lineáris	lin. gyorsulás	csukló	csuklógyorsulás
Precizios_lassu	10 mm/s	50 mm/s ²	10 °/s	40 °/s ²
Precizios_kozepes	50 mm/s	250 mm/s ²	30 °/s	100 °/s ²
Precizios_gyors	200 mm/s	1000 mm/s ²	90 °/s	300 °/s ²
Precizios_gyors_lekerekített (kiegészítő)	200 mm/s	1000 mm/s ²	90 °/s	300 °/s ² , 15 mm lekerekítés csak a _felett pontokban

3.2 Lépésről lépésre

- File → New Station**, mentés: Feladat3.rdk . Töltsd be a **KUKA LBR iiwa 7 R800**-at, és húzd be a szkripteket. 📎
- Szerszám: Program → Add Empty Tool → TCP Z = 180 → Muszer** .
- Keret:** a robot bázisa alá **Add Reference Frame → Muteti_terulet** , X = 500.
- Műtőasztal és tiltott zóna:** jelöld ki a keretet → doboz_letrehozasa kétszer: Mutoasztal (300 × 300 × 20, hely (0; 0; -20)) és **Tiltott_zona** (60 × 60 × 60, hely (0; -60; 0), *piros, átlátszó*). A névben szerepeljen a „Tiltott_zona”

szó, mert ebből tudja a mérőszkript, hogy ez a tiltott terület. 🚫 „Műteti terület, műtőasztal és a 60 × 60 × 60 mm-es tiltott zóna”.

5. Ütközésvizsgálat beállítása:

- **Tools → Check collisions:** bekapcsolás.
- **Tools → Collision map** (Shift+X): a robot és a **Muszer** sora és a **Tiltott_zona** oszlopa legyen bekapcsolva (dupla katt a cellára).
- A **Muszer** – **Műtőasztal** párt **kapcsold ki**. A célpontok az asztal felületén vannak, ezért az érintés nem ütközés.

6. **Célpontok:** aktív szerszám **Muszer**, referencia **Muteti_terulet**. Minden pont: koordináták beírása → Ctrl+T → F2. 🚫

7. **Program Precizios_lassu:** Add Program, Set Speed (lásd a táblázatot), majd a mozgások a fenti sorrendben. 🚫

8. **Az akadálykerülés bemutatása (ajánlott):** készíts egy **Precizios_HIBAS** programot, amely a C1-ből közvetlenül, alacsonyan (MoveL C1 → MoveL C2) megy át. **Shift+F5:** a RoboDK ütközést jelez, és a zóna pirosan kiemelődik.

🚫 Ez a kép bizonyítja, hogy a tiltott zóna figyelése működik. Utána a programot töröld, vagy nevezd át a nevével x_-szel kezdve, hogy ne kerüljön a mérésbe.

9. **Sebességváltozatok:** másolással **Precizios_kozepes** és **Precizios_gyors** (a Set Speed átírásával). Opcionálisan **Precizios_gyors_lekerekitett:** minden MoveL C*_felett elé **Set Rounding 15 mm**, minden célponthoz (C1, C2, C3, Kiindulas) vezető MoveL elé **Set Rounding –1**.

10. **Ismételhetőség:** futtasd ugyanazt a programot kétszer, és mindkét futás után mérd (a második mérésnek adj másik nevet). A jegyzőkönyv a **Mérések összehasonlítása** alfejezetben egymás mellé teszi a két mérést.

11. Ellenőrzés (Shift+F5) és mentés.

3.3 Hol található a mérendő adatokat?

Mérendő adat	Hol van
Az egyes pontok elérésének pontossága	Célpontok elérése táblázat, Eltérés oszlop (C1, C2, C3)
A teljes feladat végrehajtási ideje	Becsült ciklusidő
A végrehajtó szerv maximális eltérése a kívánt pozíciótól	Legnagyobb eltérés a megálló célpontokban és Legnagyobb eltérés az egyenes pályától
A sebességbeállítások hatása a pontosságra	3.7 Programok összehasonlítása (táblázat + megállapítások)
A tiltott zóna elkerülése	Legkisebb távolság a tiltott zónától (TCP), a TCP távolsága a tiltott zónától grafikon és az Ütközésvizsgálat
Ismételhetőség	Mérések összehasonlítása (két mérés), valamint Ugyanazon célpont ismételt elérésének eltérése

3.4 Várható eredmények és értelmezésük

- **Pontosság:** a C1–C3 eltérés minden sebességnél a mérési felbontáson belül, gyakorlatilag 0 mm marad (lásd 0.5). Lekerekítéssel a célpontok pontosak maradnak, mert ott nincs lekerekítés, de a biztonsági pontoknál a pálya levágja a sarkot. Ez **csökkentheti a zónától mért távolságot**, ezért nézd meg a zónagrafikont.
- **Idő:** a lassú (10 mm/s) változat sokszorosa a gyorsnak. Kis távolságoknál a gyorsítás miatt a gyors változat sem éri el mindenhol a 200 mm/s-ot (ezt a TCP-sebesség grafikonon látod).
- **Tiltott zóna:** a TCP legkisebb távolsága 40 mm körül lesz (a biztonsági magasság mínusz a zóna magassága). A műszer teste fölötté van. Az ütközésvizsgálat egyik változatnál sem jelezhet ütközést.
- **Következtetés:** szimulációban a sebesség csak az időt befolyásolja, a pontosságot nem. Valódi precíziós (orvosi) robotnál viszont a lassú, kis gyorsulású mozgás csökkenti a dinamikai hibákat és a túllendülést, ezért a célpontok közelében lassítani kell. Ez indokolja a függőleges, lassú megközelítést.

4. Mérés és jegyzőkönyv

4.1 Mérés (mindhárom állomásban)

1. Nyisd meg az állomást, és állítsd be a 3D nézetet.
2. Dupla katt a `meresek_rogzitese`-re, add meg a mérés nevét, és várd meg az összefoglaló üzenetet.
3. A `<állomás>_dokumentacio\meres_...` mappában megjelennek a CSV-k, a képek és a `meresek.json`.

A szkript a mérés előtt a robotot minden program első célpontjába állítja, így a ciklusidő nem függ attól, hol állt korábban a robot. A végén visszaállítja.

4.2 Jegyzőkönyv: egy paranccsal mindhárom feladat

A `cella_epito.py` a feladat szövegét a `feladat.txt`-be írja. Kézi útnál másold a feladat szövegét a `FeladatN_dokumentacio\feladat.txt` fájlba: az első sor a cím, a többi a szöveg, a `-` kezdetű sorokból felsorolás lesz. Ezután:

```
cd C:\RoboDK_feladatok\eszkozok
python dokumentacio\jegyzokonyv_keszito.py ..\Feladat1_dokumentacio ..\Feladat2_dokumentacio ..\Feladat3_dokumentacio --nev "Vezetéknév Keresztnév" --neptun ABC123 --targy "Tantárgy neve" --intezmeny "Egyetem, kar" --oktato "Oktató neve" --cim "RoboDK mérési feladatok" -o ..\jegyzokonyv.docx --pdf
```

A `--reszletes` kapcsolóval programonként a csuklósebesség- és csuklótartomány-grafikon is bekerül.

4.3 Befejezés Wordben

1. A sárga **[KITÖLTENDŐ]** részek (Ctrl+F: KITÖLTENDŐ):
 - Bevezetés: a gyakorlat célja (2–3 mondat).
 - Az állomás felépítése: miért ezt a robotot, szerszámot és elrendezést választottad.
 - Értékelés (feladatonként): a 6. fejezet gondolatmenete alapján, a **saját számaiddal**.
 - Összefoglalás: mit valósítottál meg, mik a fő eredmények, mit tanultál.
2. A megnyitáskor feltett kérdésre („frissíti a mezőket?”) válaszolj **Igen**-nel, ekkor a tartalomjegyzékbe bekerülnek az oldalszámok.
3. **Fájl → Mentés másként → PDF**.
4. *Ajánlott melléklet:* jobb katt egy programra → **Export Simulation** (Ctrl+E) → 3D HTML vagy 3D PDF. Ezt az oktató a böngészőben körbe tudja forgatni.

4.4 Ellenőrzőlista beadás előtt

- ☐ Mindhárom állomás el van mentve (`.rdk`), és minden program hibátlanul fut (F5, Shift+F5).
- ☐ Mindegyik feladathoz van lépésnapló (legalább 5–6 lépés, képpel).
- ☐ A mérés a végleges programokon készült. A hibás próbaprogramok neve `x_`-szel kezdődik, vagy törölted őket.
- ☐ Nincs több sárga **[KITÖLTENDŐ]** a Word-fájlban.
- ☐ Az értékelésben a saját mért számaid szerepelnek, és megmagyaráztad a szimuláció korlátait (0.5).
- ☐ A PDF elkészült, a tartalomjegyzékben vannak oldalszámok.

5. Gyakori hibák

Hiba	Megoldás
A robot nem éri el a pontot (a robot ablakában nem mozdul, vagy hibát jelez)	Tedd közelebb a keretet a robothoz (X csökkentése), vagy emeld a pontokat. A <code>cella_epito.py</code> ezt automatikusan megkeresi.
A MoveL-re szingularitást vagy csuklóhatárt jelez	Ahol nem kötelező az egyenes, cseréld MoveJ-re (jobb klikk → Joint Move), vagy változtass a célpont helyén.
A kocka nem mozdul a megfogóval	Hiányzik az <i>Attach object</i> esemény, vagy túl messze van a TCP a kockától (alapból 200 mm a tűrés).
A második futtatásnál a kocka rossz helyről indul	A program elejére tegyél <i>Simulation Event</i> → <i>Set object position (relative)</i> utasítást (1.2 / 8. lépés 1. sora).
Ütközést jelez, pedig csak érintés van (toll a papíron, műszer az asztalon)	Tools → Collision map : kapcsold ki az adott párt.
Nem készül képernyőkép	Mentsd el kézzel a szkript által kiírt néven (Win+Shift+S). A jegyzőkönyvbe így is bekerül.
A jegyzőkönyv-készítő nem talál mérést	A <code>meres_*</code> mappát keresi a megadott <code>..._dokumentacio</code> mappában. Ellenőrizd az elérési utat.
Túl hosszú a jegyzőkönyv	Csak a végleges programok legyenek a mérésben. A próbaprogramok nevét kezd <code>x_</code> -szel, és töröld vagy mozgasd át a régi <code>meres_*</code> mappákat.

6. Mit írnak az értékelésbe?

Az értékelés legyen a **te mérésed** értelmezése. Az alábbi kérdésekre válaszolj, a jegyzőkönyv számaira hivatkozva (pl. „a 3. táblázat szerint...”).

1. feladat

1. Mennyi volt a felvételtől a lerakásig eltelt idő a három sebességnél? Arányosan csökkent-e? Ha nem, miért (gyorsítás, lassítás, várakozások)?
2. Melyik volt gyorsabb, a MoveJ vagy a MoveL, és mennyivel (s, %)? Melyik tett meg rövidebb utat? Miért?
3. Mekkora volt a TCP legnagyobb sebessége? MoveJ-nél miért lehet nagyobb a beállított lineáris sebességnél?
4. Teljesül-e a sikerességi feltétel (ütközés nélkül, pontosan, ismételhetően)? Mit mutatnak ehhez a mérések, és mi lenne más valódi robotnál?

2. feladat

1. Mennyi ideig tartott a négy oldal lassan és gyorsan? Miért nem tízedére csökkent az idő a tízszeres sebességnél?
2. Mekkora volt a kezdő- és a végpozíció (P1) eltérése? Mit jelent ez az ismételhetőségre?
3. Mely tengelyek dolgoztak legtöbbször az egyes oldalakon (ΔJ -táblázat)? Miért éppen azok?
4. Okozott-e a nagyobb sebesség nagyobb pozicionálási eltérést a szimulációban? Mi okozott eltérést (lekerekítés)? Mi történne egy valódi robotnál?

3. feladat

1. Milyen pontosan érte el a robot a C1–C3 pontokat lassan és gyorsan? Mi a mérési felbontás?
2. Mennyi volt a teljes végrehajtási idő a sebességváltozatoknál?
3. Milyen közel ment a TCP a tiltott zónához (mm, mikor)? Jelzett-e ütközést a RoboDK? Hogyan oldottad meg az akadálykerülést (biztonsági magasság, függőleges megközelítés)?
4. Hogyan befolyásolja a sebesség a precíziós mozgást a szimulációban és a valóságban? Mit javasolnál egy valódi orvosi robotnál (lassú megközelítés, kis gyorsulás, nagyobb biztonsági távolság)?

Hasznos RoboDK-dokumentáció

- [3D Navigation](#): egérkezelés
- [Getting Started](#): állomás, robot, szerszám, célpont, program
- [Program menü](#): Teach Target, Add Reference Frame, Move Joint/Linear, Add Empty Tool
- [Robot panel](#): a robot pontos helyzetbe állítása
- [Program instructions](#): Set Speed, Set Rounding, Pause
- [Simulation event](#): Attach/Detach object, Set object position
- [Collision Detection](#) és [Collision Map](#)
- [Export simulation](#): 3D HTML és 3D PDF